1. Define a class calculation to implement method overloading for addition of two integers and two double variables. [5]

[Previous year paper Question: May 2025]

Definition

When **multiple methods have the same name** but **different parameters** in the same class, it is called **method overloading**.

```
public class Calculation
{

   // Method to add two integers
   void add(int a, int b)
   {
      int c;
      c=a+b;
      System.out.println("Addition of integers: " +c);
   }

   // Method to add two double values
   void add(double a, double b)
   {
      double sum;
      sum=a+b;
      System.out.println("Addition of doubles: " +sum);
   }

   public static void main(String[] args)
   {

      Calculation obj = new Calculation();

      obj.add(10,20);       // calls int method
      obj.add(10.23,23.456);  // calls double method
   }
}
```

Output:

Addition of integers: 30

Addition of doubles: 33.686

Rules of Method Overloading

1. Method name must be same
2. Parameter list must be different
3. Can have different return types
4. Can be overloaded in same class or subclass

2. Write a function using lambda expression to calculate power of number.(x^y) *[5 marks]*

[Previous year question paper May 2025]

What is Lambda Expression?

- A **Lambda expression** is a short way to write anonymous functions .
- anonymous functions means function without name.
- It is used to provide implementation of **functional interfaces**.

- **Functional Interface**

Lambda works only with interface having **one abstract method.**

- Lambda expression is a concise way to represent a method without name using -> arrow symbol.

Syntax:

(argument list) -> {body of the expression}

- **Argument List:** Parameters for the lambda expression
- **Arrow Token (->):** Separates the parameter list and the body
- **Body:** Logic to be executed.

Code:

```
interface Power
{
   double calculate(int x, int y);
}

public class Test
```

```
{
   public static void main(String[] args)
   {

      // Lambda expression
      Power p = (x, y) -> Math.pow(x, y);

      // Calling function
      System.out.println("Power is: " + p.calculate(2, 3));
   }
}
```

Output:
Power is: 8.0

Explanation

1. Interface Power

Defines method calculate(x, y)

2. Lambda Expression

(x, y) -> Math.pow(x, y)

Takes two inputs and returns power

3. Math.pow()

Built-in method to calculate power

4. Calling method

p.calculate(2,3) → $2^3 = 8$

3. What is garbage collection? Explain with required method. [5]

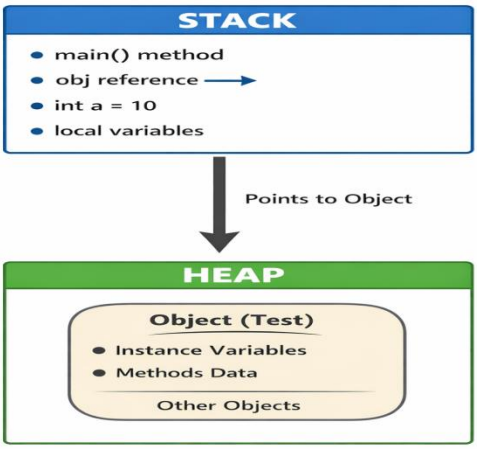
[Previous year question paper May 2025]

➢ What is Garbage Collector (GC)?

- Garbage Collector is a **memory management system** in Java that automatically removes unused objects from memory.
- It frees memory by deleting objects that are **no longer referenced** by any part of the program.
- This helps prevent **memory leaks** and improves performance.

Definition:

" Garbage Collection is a process that automatically destroys unused objects to free heap memory."



Real-Life Example

- Think of your computer desktop
- When files are not needed, you delete them to free space.

- Garbage Collector does the same with unused objects in memory.
-

Why Garbage Collection is Needed

- To free unused memory
- To avoid memory leaks
- To improve program performance
- To manage heap memory automatically
- No need for manual memory deletion

How Garbage Collector Works

- Java automatically checks objects in **Heap Memory**
- If an object has:

❌No reference
❌Not used anywhere

GC removes it and frees memory

Example of Garbage Collection

```
class Test {
   public static void main(String[] args) {

       Test t1 = new Test();
       Test t2 = new Test();

       t1 = null;  // eligible for GC
       t2 = null;  // eligible for GC

       System.gc(); // request GC
   }
}
```

Since objects have no reference → GC can remove them

Required Methods Related to Garbage Collection

1. System.gc()

- Requests JVM to run Garbage Collector
- It is NOT guaranteed to run immediately
- System.gc();

2. Runtime.getRuntime().gc()

- Another way to request GC

Runtime.getRuntime().gc();

3. finalize() Method

- Called by GC before destroying object
- Used to perform cleanup tasks
- **Example:**

```
class Test1
{

   protected void finalize()
   {
      System.out.println("Object destroyed");
   }

   public static void main(String[] args)
   {
      Test t = new Test();
      t = null;
      System.gc();
   }
}
```

4.Write a java program to demonstrate how to create user-defined exception.

[Previous year Question Peper: 5 Marks]

- Java allows programmers to **create their own custom exceptions**.
- These are called **User Defined Exceptions**.
- A user-defined exception is created by **extending the Exception class**.

Steps to Create User Defined Exception

1. Create a class that **extends Exception**.
2. Create a **constructor** to display the error message.
3. Use the **throw keyword** to throw the exception.
4. Handle it using **try-catch block**.

Syntax

```
class MyException extends Exception
{
   MyException(String message)
   {
      super(message);
   }
}
```

super(message) is used to pass the message to the Exception class.

Example

```
class MyException extends Exception
{
    MyException(String msg)
    {
        super(msg);
    }
}
```

```
class Test6
{
    public static void main(String args[])
    {
        try
        {
            throw new MyException("This is custom
exception");
        }
        catch(MyException e)
        {
            System.out.println(e.getMessage());
        }
    }
}
```

Output

This is custom exception

Explanation:

1. MyException is a **user defined exception class** that extends the Exception class.
2. The constructor passes the **error message** using super(msg).
3. In main(), the program **throws the custom exception** using throw.
4. The catch block **handles the exception** and prints the message using e.getMessage().

5. Create a thread to display prime numbers between 1 to 500 each number will display after 3 seconds. **[5]**

Program Code:

```
class MyThread extends Thread
{
    public void run()
    {
        for(int i=2;i<=500;i++)
        {
            int flag=0;

            for(int j=2;j<i;j++)
            {
                if(i%j==0)
                {
                    flag=1;
                    break;
                }
            }

            if(flag==0)
            {
                System.out.println(i);

                try{
                    Thread.sleep(3000);   // 3 seconds delay
                }
                catch(Exception e){}
```

```
            }
        }
    }
}

class DemoPrime
{
    public static void main(String args[])
    {
        MyThread t = new MyThread();
        t.start();
    }
}
```

Output:

2
3
5
7
11
13
17
19
23
29
…

Explanation:

- This program creates a **thread by extending the Thread class**.
- The run() method checks numbers from **2 to 500** to find **prime numbers**.
- If a number is prime, it is printed using System.out.println().
- After printing each prime number, the thread pauses for **3 seconds using** Thread.sleep(3000).
- The thread starts execution using the start() **method** in the main() function.